

CSC311 Introduction to Machine Learning Week 5 & 6 Exercises

When submitting problems, please **clearly indicate which question you are submitting**. Use the full question number shown below.

Training

W6-T1. Consider the computation $\mathbf{y} = \text{softmax}(\mathbf{z})$. Compute the derivative $\frac{\partial y_i}{\partial z_j}$.

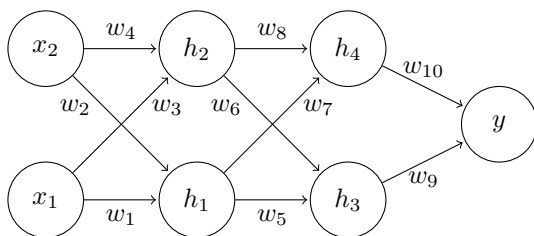
W6-T2. Consider the following data set

$$\mathbf{X} = \begin{bmatrix} 2 & 0 \\ 1 & -1 \\ 2 & 1 \\ 0 & 1 \end{bmatrix} \quad \mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Is there a value of $\mathbf{w}$ for which the model $y = \sigma(\mathbf{w}^T \mathbf{x})$ perfectly classifies all 4 data points? If your answer is “yes”, provide such a value. If your answer is “no”, provide a proof.

W6-T3. Consider the computation $\mathbf{y} = \text{ReLU}(\mathbf{z})$. Compute the derivative $\frac{\partial y_i}{\partial z_j}$.

W6-T4. Consider the network shown in the below figure, where none of the units have bias parameters. All the hidden units use the ReLU activation function, so that at each hidden unit, $h_j = \text{ReLU}(m_j)$, where m_j is the total input to unit h_j . The output unit uses the sigmoid activation function, so that $y = \sigma(z)$, where z is the total input to unit y . We are trying to minimize a loss function $\mathcal{L}(y, t)$, where t is the target value, using gradient descent.



- Is this a computation graph above? Why or why not?
- Write down the expression for how the error signal $\overline{w_6}$ is computed, in terms of previously computed quantities, during back propagation.
- Write down the expression for how the error signal $\overline{h_2}$ is computed, in terms of previously computed quantities, during back propagation.
- The activation value h_2 is *not* a parameter that we learn. Explain why it is still useful to compute the error signal $\overline{h_2}$.

W6-T5. Suppose that we have a 2-layer neural network, where the threshold activation function is used in the hidden layer, and the softmax activation function is used in the final layer.

- What is the prediction for the data point $\mathbf{x} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ if we have the following weights and biases?

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1 & 1 & 1 \\ 1 & -1 & 0 \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- Clearly identify which quantity above represent the value of the input layer, the hidden layer, and the output layer.

W6-T6. What is the difference between a weight (i.e., a parameter) and a hidden activation (i.e., the value of a hidden unit)? Which of these is learned using gradient descent? When making predictions about two test data points $\mathbf{x}^{(1)}$ and $\mathbf{x}^{(2)}$, which of these stays the same and which of these take on different values for each data point?

W6-T7. We discussed the following activation functions: ReLU, sigmoid, and hard threshold functions.

- (a) Why can't the hard threshold function be used in a neural network trained with gradient descent?
- (b) Which activation function do practitioners generally use first in the hidden layers?
- (c) Why is it that in the output layer, practitioners generally use either the sigmoid, softmax, or identity activation functions?
- (d) The ReLU, sigmoid and tanh activation functions can produce poor gradient signal at certain input values. What input values are these, and why?

W6-T8. Consider the computation below, where each of $\mathbf{d}$, $\mathbf{e}$, $\mathbf{f}$, are column vectors, and $\mathbf{C}$ is a matrix. We will be computing the gradient $\nabla_{\mathbf{d}}\mathcal{L}$ step by step, using a method called *backpropagation* that will be explored in more detail in the next lecture.

$$\begin{aligned}\mathbf{e} &= \mathbf{C}\mathbf{d} \\ \mathbf{f} &= \sigma(\mathbf{e}) \\ \mathcal{L} &= \mathbf{f}^T(\mathbf{f} + \mathbf{e})\end{aligned}$$

- (a) Compute the scalar derivative $\frac{\partial \mathcal{L}}{\partial f_i}$. Then, write down the gradient $\nabla_{\mathbf{f}}\mathcal{L}$ using only vector quantities
- (b) Compute the scalar derivative $\frac{\partial f_i}{\partial e_j}$. You may use the fact that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ without proof. Recall that $\sigma(\mathbf{e})$ is applied element-wise.
- (c) Compute the scalar derivative $\frac{\partial \mathcal{L}}{\partial e_j}$. Then, write down the gradient $\nabla_{\mathbf{e}}\mathcal{L}$ using only vector quantities.
- (d) Compute the scalar derivative $\frac{\partial e_j}{\partial d_k}$.
- (e) Compute the scalar derivative $\frac{\partial \mathcal{L}}{\partial d_k}$. Then, express the gradient $\nabla_{\mathbf{d}}\mathcal{L}$ using only vector quantities

W6-T9. Suppose that we have a neural network with the following forward pass computation (for a single data point):

$$\begin{aligned}\mathbf{m} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \\ \mathbf{h} &= \text{ReLU}(\mathbf{m}) \\ \mathbf{z} &= \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)} \\ \mathbf{y} &= \text{softmax}(\mathbf{z}) \\ \mathcal{L} &= \mathcal{L}_{CE}(\mathbf{y}, \mathbf{t})\end{aligned}$$

We will derive the backward pass computation for a single data point.

- (a) Show that $\bar{\mathbf{z}} = \mathbf{y} - \mathbf{t}$
- (b) Show that $\overline{\mathbf{W}^{(2)}} = \bar{\mathbf{z}}\mathbf{h}^T$
- (c) Show that $\overline{\mathbf{b}^{(2)}} = \bar{\mathbf{z}}$
- (d) Show that $\bar{\mathbf{h}} = \mathbf{W}^{(2)T}\bar{\mathbf{z}}$
- (e) Show that $\bar{\mathbf{m}} = \bar{\mathbf{h}} \circ \text{ReLU}'(\mathbf{m})$
- (f) Show that $\overline{\mathbf{W}^{(1)}} = \bar{\mathbf{m}}\mathbf{x}^T$
- (g) Show that $\overline{\mathbf{b}^{(1)}} = \bar{\mathbf{m}}$

W6-T10. Consider this computation, where each of x , z , y , and t are scalars.

$$\begin{aligned}z &= wx + b \\y &= x + \tanh(z) \\ \mathcal{L} &= \frac{1}{2}(y - t)^2\end{aligned}$$

- (a) Draw a computation graph consisting of the quantities above.
- (b) Compute the derivative of the tanh activation function $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$.
- (c) Compute the following derivatives using the backpropagation algorithm. Show all your work.

$$\begin{aligned}\overline{\mathcal{L}} &= 1 \\ \overline{y} &= \\ \overline{z} &= \\ \overline{w} &= \\ \overline{b} &= \\ \overline{x} &= \end{aligned}$$

Validation

W6-V1. Consider the following model that performs multi-class classification, where the inputs $\mathbf{x} \in \mathbb{R}^2$, and the ground truth $\mathbf{t} \in \mathbb{R}^3$.

$$\begin{aligned}\mathbf{z} &= \mathbf{W}\mathbf{x} + \mathbf{b} \\ \mathbf{y} &= \text{softmax}(\mathbf{z})\end{aligned}$$

However, some weights in $\mathbf{W}$ are tied, and the values $a, c, d \in \mathbb{R}$ appear in multiple components of $\mathbf{W}$:

$$\mathbf{W} = \begin{bmatrix} a & c \\ c & a \\ d + c & d - c \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

We would like to train this neural network using the cross-entropy loss.

- (a) Is $\mathbf{z}$ a vector, matrix, or a scalar? What is the shape (dimension) of the quantity $\mathbf{z}$?
- (b) Write down the gradient $\nabla_{\mathbf{y}}\mathcal{L}$, and $\nabla_{\mathbf{z}}\mathcal{L}$, where $\mathcal{L}$ is the cross-entropy loss.
- (c) Compute the derivative $\frac{\partial \mathcal{L}}{\partial a}$.

W6-V2. Suppose that we have a neural network with the following forward pass computation (for a single data point):

$$\begin{aligned}\mathbf{m} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \\ \mathbf{h} &= \text{ReLU}(\mathbf{m}) \\ \mathbf{z} &= \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)} \\ \mathbf{y} &= \text{softmax}(\mathbf{z}) \\ \mathcal{L} &= \mathcal{L}_{CE}(\mathbf{y}, \mathbf{t})\end{aligned}$$

We showed, during our lectures and above, that the backward pass (for that single data point) can be computed as follows:

$$\begin{aligned}
\bar{\mathbf{z}} &= \mathbf{y} - \mathbf{t} \\
\overline{\mathbf{W}^{(2)}} &= \bar{\mathbf{z}} \mathbf{h}^T \\
\overline{\mathbf{b}^{(2)}} &= \bar{\mathbf{z}} \\
\bar{\mathbf{h}} &= \mathbf{W}^{(2)T} \bar{\mathbf{z}} \\
\bar{\mathbf{m}} &= \bar{\mathbf{h}} \circ \text{ReLU}'(\mathbf{m}) \\
\overline{\mathbf{W}^{(1)}} &= \bar{\mathbf{m}} \mathbf{x}^T \\
\overline{\mathbf{b}^{(1)}} &= \bar{\mathbf{m}}
\end{aligned}$$

For the remaining parts of this question, we will be vectorizing the backward pass computation for a batch of data. We will use the following notation:

- Scalars N to represent the batch size, D to present the number of features, L to represent the number of hidden units, K to represent the number of classes.
- $\mathbf{X}$ to represent the $N \times D$ data matrix.
- $\mathbf{M}$ and $\mathbf{H}$ to represent the $N \times L$ matrix of pre- and post-ReLU hidden activations.
- $\mathbf{Z}$ and $\mathbf{Y}$ to represent the $N \times K$ matrix of pre-softmax logits and post-softmax predictions.
- $\mathbf{T}$ to represent the $N \times K$ matrix of ground-truth
- $\mathbf{W}^{(1)}$ has shape $L \times D$, $\mathbf{W}^{(2)}$ has shape $K \times L$

Moreover, we will use the notation $\mathbb{1}$ to represent a vector consisting of all 1s of shape $N \times 1$.

Thus, the vectorized forward pass for our neural network becomes:

$$\begin{aligned}
\mathbf{M} &= \mathbf{X}(\mathbf{W}^{(1)})^T + \mathbb{1}(\mathbf{b}^{(1)})^T \\
\mathbf{H} &= \text{ReLU}(\mathbf{M}) \\
\mathbf{Z} &= \mathbf{H}(\mathbf{W}^{(2)})^T + \mathbb{1}(\mathbf{b}^{(2)})^T \\
\mathbf{Y} &= \text{softmax}(\mathbf{Z}) \\
\mathcal{E} &= \frac{1}{N} \sum_i \mathcal{L}_{CE}(\mathbf{y}^{(i)}, \mathbf{t}^{(i)})
\end{aligned}$$

Where $\mathbf{y}^{(i)}$ and $\mathbf{t}^{(i)}$ index a row in $\mathbf{Y}$ and $\mathbf{T}$.

We will be computing gradients with respect to the cost $\mathcal{E}$, so that notations like $\bar{\mathbf{Z}}$ below are slightly abused to represent vectorized **cost-derivatives**, i.e. $\bar{\mathbf{Z}}$ represents the vector containing entries $\frac{\partial \mathcal{E}}{\partial z_j^{(k)}}$ (as opposed to **loss-derivatives** $\frac{\partial \mathcal{L}}{\partial z_j}$ computed during class).

- Show that $\bar{\mathbf{Z}} = \frac{1}{N}(\mathbf{Y} - \mathbf{T})$.
- Show that $\overline{\mathbf{W}^{(2)}} = \bar{\mathbf{Z}}^T \mathbf{H}$.
- Show that $\overline{\mathbf{b}^{(2)}} = \bar{\mathbf{Z}}^T \mathbb{1}$, where $\mathbb{1}$ is the $N \times 1$ vector of 1's.
- Show that $\bar{\mathbf{H}} = \bar{\mathbf{Z}} \mathbf{W}^{(2)}$.
- Show that $\bar{\mathbf{M}} = \bar{\mathbf{H}} \circ \text{ReLU}'(\mathbf{M})$, where $\circ$ represents element-wise multiplication, and ReLU' is the derivative of the ReLU activation function.
- Argue, using your earlier results, that $\overline{\mathbf{W}^{(1)}} = \bar{\mathbf{M}}^T \mathbf{X}$ and $\overline{\mathbf{b}^{(1)}} = \bar{\mathbf{M}}^T \mathbb{1}$.

W6-V3. Consider the following neural network that computes the representations of two inputs, and then compares their representations. This neural network architecture is sometimes called the **twin** neural network.

$$\begin{aligned}\mathbf{h}^{(a)} &= \Psi(\mathbf{x}^{(a)}) \\ \mathbf{h}^{(b)} &= \Psi(\mathbf{x}^{(b)}) \\ y &= \text{dist}(\mathbf{h}^{(a)}, \mathbf{h}^{(b)})\end{aligned}$$

Here, Ψ computes the representation of an input, and dist compares two representations. In our case, Ψ and dist will both be neural networks with the following architectures:

$$\begin{aligned}\Psi(\mathbf{x}) &= \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \\ \text{dist}(\mathbf{h}^{(a)}, \mathbf{h}^{(b)}) &= \sigma(\mathbf{W}^{(2)}(\mathbf{h}^{(a)} \odot \mathbf{h}^{(b)}) + b^{(2)})\end{aligned}$$

Where $\mathbf{h}^{(a)} \odot \mathbf{h}^{(b)}$ denotes element-wise multiplication: $(\mathbf{h}^{(a)} \odot \mathbf{h}^{(b)})_j = h_j^{(a)} \cdot h_j^{(b)}$

We will use the binary cross-entropy loss $\mathcal{L} = -t \log y - (1 - t) \log(1 - y)$ to train this neural network.

(a) Suppose, for this part only, that

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0.5 & -0.5 \\ 0 & 1 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \mathbf{0}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & 1 \end{bmatrix}, \quad b^{(2)} = 0$$

Compute the prediction y for the following inputs $\mathbf{x}^{(a)}$ and $\mathbf{x}^{(b)}$

$$\mathbf{x}^{(a)} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{x}^{(b)} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

(b) For the rest of this question, let's introduce some intermediate variables to our computation to make backpropagation a bit easier.

$$\begin{aligned}\mathbf{m}^{(a)} &= \mathbf{W}^{(1)}\mathbf{x}^{(a)} + \mathbf{b}^{(1)} \\ \mathbf{h}^{(a)} &= \text{ReLU}(\mathbf{m}^{(a)}) \\ \mathbf{m}^{(b)} &= \mathbf{W}^{(1)}\mathbf{x}^{(b)} + \mathbf{b}^{(1)} \\ \mathbf{h}^{(b)} &= \text{ReLU}(\mathbf{m}^{(b)}) \\ \mathbf{h} &= \mathbf{h}^{(a)} \odot \mathbf{h}^{(b)} \\ z &= \mathbf{W}^{(2)}\mathbf{h} + b^{(2)} \\ y &= \sigma(z) \\ \mathcal{L} &= -t \log y - (1 - t) \log(1 - y)\end{aligned}$$

Using backpropagation, derive the equations for computing $\overline{\mathbf{b}^{(2)}}$ and $\overline{\mathbf{W}^{(2)}}$.

(c) Using backpropagation, derive the equations for computing $\overline{\mathbf{h}}$, $\overline{\mathbf{h}^{(a)}}$ and $\overline{\mathbf{h}^{(b)}}$.

(d) Using backpropagation, derive the equations for computing $\overline{\mathbf{W}^{(1)}}$ and $\overline{\mathbf{b}^{(1)}}$.

Generalization

Recall that the “Generalization” questions are beyond the scope of this course, but are intended to challenge you and expand your understanding of the materials. For this set of Generalization questions, in particular, we will ask you to do some research on your own about concepts that are important to the practical use of neural networks.

1. What is the “vanishing gradient problem”? Explain how ReLU units help to mitigate this problem.
2. What is a “dead unit” in a neural network? Explain how a “dead ReLU” problem could occur. Explain how “Leaky ReLU” units can help to mitigate this problem.